COMS E6424 Hardware Security
Homework 3 Reflection Report

Peiheng Li(pl2978) Peng Chen(pc3193)
Wenxuan Xu(wx2341) Xiangyu Liao(xl3581) Yinfeng Chai(yc4669)

March 29, 2026

1 Overview

This homework asked us to build a covert channel between two user processes running on the same VM without using an overt communication path. Our final implementation is a Flush+Reload channel built around the shared `libc` symbol `ecvt_r`. The sender keeps that function hot in the instruction cache when it wants to transmit a `1`, and it repeatedly flushes the relevant cache lines when it wants to transmit a `0`. The receiver probes the same function, measures the access time with `RDTSCP`, and decides which bit was present in the current slot.

The appendix provided a useful starting point for the basic attack primitive, but that alone was not enough for a reliable communication channel. Most of the work went into making the signal usable under realistic noise. In the final version, we added fixed-duration time slots, a short frame format with a synchronization word and a payload length, and a receiver-side decision rule based on the aggregate behavior of many probes inside each slot.

2 Techniques We Tried and Why

The first version was essentially the appendix receiver: time a single call to `ecvt_r`, compare it against a threshold, and print `0` or `1`. That confirmed the microarchitectural effect was real, but the output was not useful as a communication channel. A raw bit stream is hard to align. It is even harder to decode when the sender and receiver start at different times.

We then moved to the current design. Both processes divide time into slots using `CLOCK_MONOTONIC`. Once they share the same bit duration, they also share the same slot boundaries, so no explicit handshake is needed. The sender computes the transmitted bit as `slot % total_bits`, while the receiver samples throughout that same slot and bases its decision on the overall hit ratio. This was a much better fit for a noisy VM than relying on a single timing measurement per bit.

Framing was the second key improvement. Each transmission begins with `0xA5 0x5A`, followed by a one-byte payload length and then the message itself. The receiver keeps a 16-bit shift register and searches for `0xA55A`. Once the sync word appears, the next eight bits are interpreted as the length byte, and the following bits are decoded as payload. This made the receiver far less sensitive to startup offset and temporary loss of alignment.

We also considered adding explicit error-correcting codes. We did not implement them in the final version because the simpler design was already adequate at the bit durations we were using. Instead, the receiver treats an ambiguous slot as a local failure, discards the partial frame, and waits for the next synchronization pattern. This reduces the chance that one uncertain decision corrupts the rest of the message.

3 Implementation Summary

The submission contains three C programs.

`threshold.c` is the calibration tool. It repeatedly times `ecvt_r` in two conditions: after flushing the relevant cache lines, and without flushing them. The purpose is to estimate a threshold that separates the two timing clusters on the machine currently being tested.

`sender.c` resolves `ecvt_r` with `dlopen/dlsym`, builds a frame, and then transmits one bit per time slot. For a `1`, it repeatedly calls `ecvt_r` until the slot ends. For a `0`, it repeatedly flushes the same code region.

`receiver.c` probes `ecvt_r` for the duration of a slot, counts how many probes fall below the configured threshold, and turns the resulting hit ratio into a bit decision. It then looks for the sync word, reads the length byte, and prints the recovered payload.

4 Threshold Tuning

On our test machine the threshold tuner consistently produced two clusters. A short calibration log is included with the submission in `report_assets/threshold_measurements.txt`. In the later portion of that log, the non-flushed average settles near 366 cycles and the flushed average near 844 cycles. We therefore used 600 cycles as the receiver threshold. The choice is deliberately conservative. It sits far enough from both means that the receiver can tolerate noise while still separating hits from misses.

The threshold is machine-dependent, which is why we left it configurable at runtime. The protocol itself does not rely on a specific cycle count. What matters is that the chosen value separates the two timing distributions reasonably well on the machine being tested.

These calibration numbers should not be read as identical to the values in Figure 1. The threshold tuner measures a single timed call after a local flush or non-flush decision. The plot, in contrast, shows slot-level average probe times while a separate sender process is actively transmitting. During a `0` slot, the sender repeatedly flushes the target throughout the entire slot, so the observed average probe time is much higher, around 1.8×10^3 cycles on our machine.

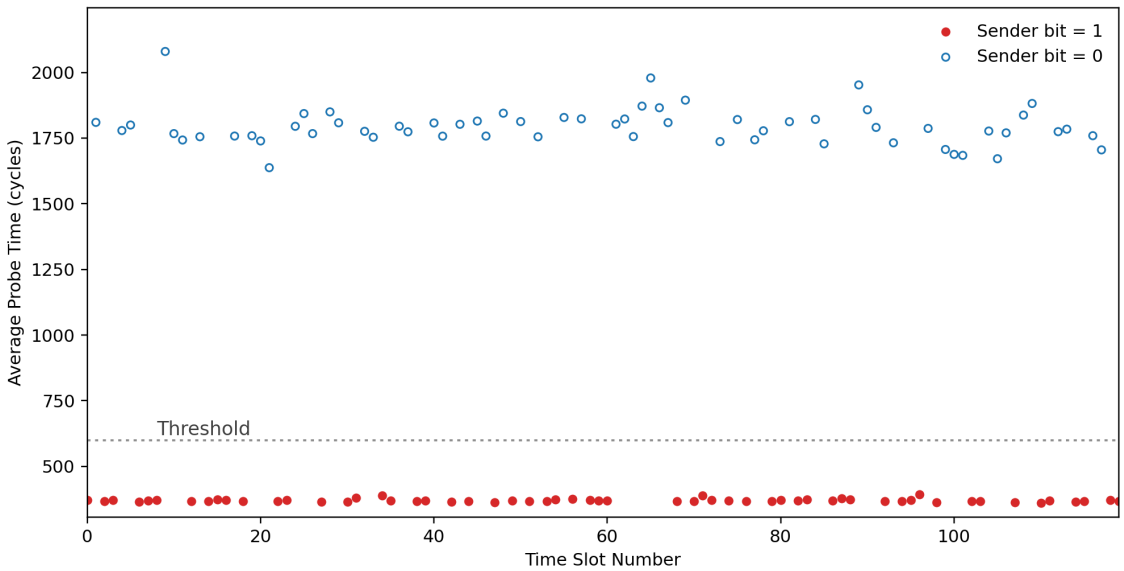

Figure 1: Probe-time measurements for `ecvt_r` over 120 consecutive time slots while the sender transmits the frame for `test123` at 5 ms per bit. The figure is inspired by the probe-time plot in the original Flush+Reload paper, but it is generated from our own implementation and our own measurements. Low points correspond to cache-hot slots, while high points correspond to slots in which the sender continuously flushes the target during transmission of a `0`.

Figure 1 shows the behavior we relied on during decoding. The timing noise is visible, which is expected in a virtualized environment, but the two groups are still clearly separated. That separation is what makes the channel practical.

5 Expected Bandwidth Versus Measured Bandwidth

For a payload of length L bytes, the frame contains $L+3$ bytes because we add two sync bytes and one length byte. The expected payload goodput is therefore

$$R_{\text{payload}} = \frac{8L}{8(L+3)d} = \frac{L}{(L+3)d},$$

where d is the bit duration in seconds. For the test string `test123`, $L = 7$, so the payload fraction of each frame is $7/10$.

We measured the channel with the current code by running the sender and receiver concurrently and counting how many exact `test123[END]` decodes appeared during a 12-second observation window. Table 1 summarizes the results.

Bit dur. (ms)	Expected goodput (bps)	Observed goodput (bps)	Exact frames in 12 s	Notes
25	28.0	23.3	5	Stable
10	70.0	65.3	14	Stable
5	140.0	121.3	26	Stable; comfortable operating point
2	350.0	336.0	72	One uncertain slot in 12 s
1	700.0	602.0	129	Visible decoding errors begin

Table 1: Measured payload goodput for the current implementation using the message `test123`. The observed rate counts only exact payload recoveries.

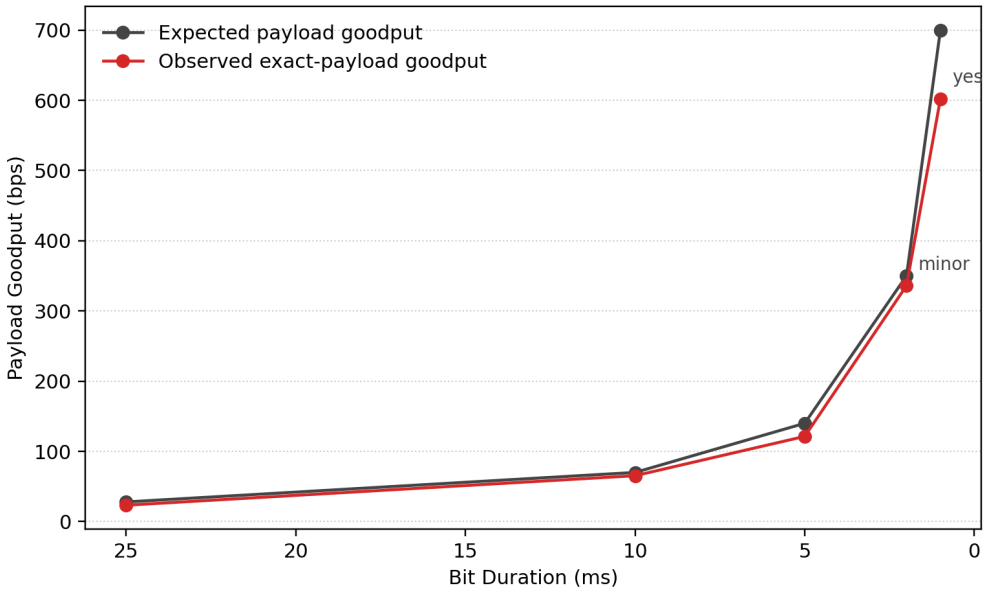

Figure 2: Expected payload goodput versus observed exact-payload goodput. The channel scales well as the bit duration decreases, but at 1 ms the receiver starts to show visible corruption.

The trend is consistent with what we expected. Shorter slots increase the raw capacity almost linearly, but they also reduce the timing margin available to the receiver. Up to about 5 ms, the current implementation remains stable on our machine. At 2 ms, it still works well, although the receiver has less margin against noise. At 1 ms, many frames are still decoded correctly, but visible corruption begins to appear. For that reason, 5 ms is the operating point we would recommend, while 2 ms is a more aggressive setting that still performs reasonably well on this VM.

Latency also improved substantially. With the older 200 ms setting from the README, one complete `test123` frame took about 20.1 s to appear in our first-message test. At 10 ms, the same message was recovered in under one second. The protocol stayed the same; the main difference was that the channel had enough signal quality to support much shorter slots.

6 Hardware and Reproducibility

Our measurements were taken on a Linux guest running under WSL2 on an AMD Ryzen 9 9950X3D system. The guest reports 32 logical CPUs, `clflush`, and `rdtscp` support. The environment

matters. This is still a VM, so there is scheduler noise and hypervisor interference, but the shared-library code pages are available to both processes, which is exactly what Flush+Reload needs.

To reproduce the basic experiment:

1. Compile with `make`.
2. Run `./threshold` and choose a threshold between the flushed and non-flushed timing clusters. On our machine, 600 cycles worked well.
3. Start the receiver, for example `./receiver -t 600 -d 5`.
4. Start the sender in another terminal, for example `./sender -m "test123" -d 5`.

The code is intentionally parameterized. If the same threshold does not work on another machine, the correct fix is to retune the threshold, not to change the protocol.

7 Reflection

The main lesson from this homework is that detecting a cache effect is only the first step. Turning that effect into a communication channel is a separate problem. A design that looks fine when inspected one probe at a time can still fail as soon as timing drift, startup misalignment, or scheduler noise enters the picture.

The final design is still relatively simple, but each part serves a clear purpose. Time slots reduce drift. The sync word establishes frame boundaries. The length byte tells the receiver when the payload is complete. Majority voting inside a slot makes the bit decision less sensitive to outliers, and the reset-on-uncertainty rule limits error propagation. None of these mechanisms is complicated on its own, but together they make the difference between a timing demo and a usable covert channel.

If we had more time, the next step would be to add lightweight forward-error correction or at least a checksum over the payload. That would be most useful near 1 ms, where the channel becomes faster but also less clean. Another reasonable extension would be CPU pinning, which could reduce scheduler interference and make the shorter bit durations more stable.

8 Conclusion

We implemented a working Flush+Reload covert channel that transmits framed messages between two local processes through shared instruction-cache state. The core microarchitectural primitive came from the assignment appendix, but the reliable channel behavior came from protocol work layered on top of it. On our test platform, the implementation is comfortable at 5 ms per bit and remains mostly usable down to 2 ms. Below that point the bandwidth keeps increasing, but the error rate starts to become visible.

References

[1] Yuval Yarom and Katrina Falkner, *Flush+Reload: A High Resolution, Low Noise, L3 Cache Side-Channel Attack*, USENIX Security Symposium, 2014.

[2] COMS E6424 Hardware Security Homework #3, Spring 2026.